



VulTrLM: LLM-assisted vulnerability detection via AST decomposition and comment enhancement

Shaobo Zhang^{1,2} · Qianzhi Wang^{1,2} · Qin Liu³ · Entao Luo⁴ · Tao Peng⁵

Received: 28 April 2025 / Accepted: 16 September 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Software vulnerability detection is a critical task in software engineering to ensure system security. Recently, pre-trained models have shown great potential in automating vulnerability detection. However, existing pre-trained model-based vulnerability detection approaches still suffer from the following issues: (1) They struggle to accurately parse the complex expressions with multiple operators and pointers. (2) They fail to capture various code execution sequences effectively. To mitigate these issues, we propose VulTrLM, a vulnerability detection framework that decomposes Abstract Syntax Trees (ASTs) with the assistance of Large Language Models (LLMs). To improve code semantics, VulTrLM leverages an improved decomposition algorithm to partition the AST into multiple subtrees, each paired with comments generated by LLMs. To enhance execution sequence sensitivity, we designed a Transformer-based semantic aggregator, which models relationships between subtrees and merges their features into a target vulnerability vector. Finally, VulTrLM incorporates the vulnerability vector to effectively capture vulnerability patterns. Experimental results showed that VulTrLM achieves a higher F1 score than baselines with improvements of 1.87% on FFMpeg+Qemu, 4.82% on Reveal, and 12.46% on SVulD. Moreover, VulTrLM can remain effective across different pre-trained models, and the ablation studies also validate the effectiveness of VulTrLM's core design.

Keywords AST decomposition · Large language model · Pre-trained model · Vulnerability detection

1 Introduction

Software vulnerabilities threaten the security of modern software systems (Zhang et al. 2021; Feng et al. 2022; Zhang et al. 2024b; Wan et al. 2024), and their exposure amplifies risks across various domains, such as endpoint devices (Zhang et al. 2025b; Liu et al.

Communicated by: Chunyang Chen and James Zheng.

This article belongs to the Topical Collection: *Special Issue on When Software Security Meets Large Language Models: Opportunities and Challenges*

Extended author information available on the last page of the article

2025b), application privacy (Peng et al. 2024; Zhang et al. 2023b; Ma et al. 2025), and decentralized systems (Deng et al. 2024; Uddin et al. 2025). According to CrowdStrike's 2024 Threat Hunting Report (CrowdStrike 2024), system security vulnerabilities are being increasingly exploited by cyber attackers, resulting in serious data security risks and potentially substantial economic losses. Moreover, the number of identified Common Vulnerabilities and Exposures (CVEs)¹ continues to escalate, with 40,300 (CVE security vulnerability database 2025) CVEs disclosed in 2024, representing a 38% increase compared with 2023. In light of this trend, the development of effective automated vulnerability detection methods has become crucial to modern software engineering.

Current automated vulnerability detection approaches can be broadly categorized into rule-based and learning-based methods. Rule-based approaches, often referred to as static or program analysis, are based on user-defined rules and pattern matching. For example, Flawfinder² detects potential vulnerabilities by scanning source code using predefined rules. These approaches work well for detecting known vulnerability patterns. However, they are heavily dependent on manual effort and domain expertise (Zhu et al. 2022), and often fail to yield accurate results in the presence of unknown vulnerabilities or complex code structures.

In search of a more effective extraction of vulnerability-related features, several studies (Russell et al. 2018; Steenhoek et al. 2023; Ni et al. 2023a) have explored learning-based methods. These approaches, particularly those using Deep Learning (DL), can automatically learn high-dimensional features from source code to identify potential vulnerabilities. For instance, Li et al. (2018) transformed source code into token sequences and trained models using recurrent neural networks, but such plain-text representations often fail to capture code structure and semantics. Consequently, recent studies (Chakraborty et al. 2021; Wu et al. 2022) have focused on refining source code into various graph representations, such as ASTs, Control Flow Graphs (CFGs), and Program Dependency Graphs (PDGs). In parallel, convolutional neural networks and Graph Neural Networks (GNNs) have been employed to help with graph learning. However, the performance of DL-based methods remains suboptimal due to the scarcity of high-quality labeled training data (Croft et al. 2023).

In response to this issue, researchers have progressively applied pre-trained models for vulnerability detection. Current methods often fine-tune pre-trained models on vulnerability-specific datasets (Deng et al. 2025b), using their general code understanding to improve detection, such as EPVD (Zhang et al. 2023a) and SVulD (Ni et al. 2023b). Although existing pre-trained model-based approaches outperform DL-based methods in vulnerability detection, they still have issues in two aspects:

Complex Logic The first issue is that pre-trained models often struggle to understand code with complex logic. Ideally, we expect that a pre-trained model should be able to parse such expressions into their correct logical semantics. However, we observe that some state-of-the-art models consistently fail to make correct predictions on vulnerable code involving multiple operators such as `||` and `->`. Thus, it is urgent to improve the semantic representation of source code so that pre-trained models can better capture vulnerability patterns involving complex logic.

¹ <https://cve.mitre.org/>

² <https://dwheeler.com/flipfinder/>

Execution Sequences The second issue is that pre-trained models often treat code as linear token sequences, ignoring its non-linear execution structure. In many cases, the activation of a vulnerability depends on multiple *if* statements and *return* statements. For example, certain vulnerabilities are only triggered when multiple *if* conditions are satisfied in a specific sequence. However, pre-trained models often fail to recognize that some code segments execute only under particular conditions, leading them to ignore vulnerabilities hidden in specific execution paths. Therefore, it is crucial to help pre-trained models better capture code execution sequences.

We analyzed these two issues and offered some insights. **The first issue** stems from the attention mechanism (Soydaner 2022) of Transformer-based pre-trained models. Despite their effectiveness in capturing local dependencies, these models often break complex pointer operations into isolated symbolic units, making it hard to grasp deeper semantic relationships. To address this issue, we consider using AST instead of code as the input representation. Unlike plain text, ASTs are much better for working with code because they keep the structure and order of operations clear. For instance, Tipirneni et al. (2024) embedded root-to-leaf AST paths into the encoder to enhance the model's structural comprehension. Besides, modern pre-trained models exhibit natural language and programming language (NL-PL) alignment (Feng et al. 2020), allowing cross-modal mapping between code and textual descriptions. It means that we can make use of LLM-generated code comments to provide extra information (Misra et al. 2020) that helps the pre-trained model better understand semantic relationships. **The second issue** comes from pre-trained models often treating code as plain sequences, which makes them miss important structures like bracket nesting and scopes. In contrast, ASTs break code into tree-like nodes (e.g., functions, loops, and conditionals), explicitly demonstrating parent-child relationships and execution paths between code blocks. Prior work Jiang et al. (2021) and Zhang et al. (2023a) has shown that using ASTs can improve structural relationships. However, as code becomes more complex, the AST structure also grows deeper and wider, making it harder to capture long-range dependencies. This motivates the need to decompose the tree into smaller, more manageable substructures.

Based on the aforementioned insights, we propose VulTrLM, a novel vulnerability detection framework that enhances pre-trained models with structural and semantic cues extracted from source code. The core idea is to combine AST decomposition with natural language comments to form a richer representation of code semantics and execution logic. Specifically, given a code snippet, VulTrLM first parses it into an AST. To alleviate the complexity of large ASTs, we designed an AST decomposition algorithm that splits the AST into a set of smaller, coherent subtrees. For each resulting subtree, VulTrLM leverages an LLM to generate a corresponding natural language comment, which is attached as a comment node to form a new structure called a Commented Abstract Subtree (CAS). Next, both the source code and the set of CASs are encoded by the pre-trained model into their corresponding embeddings. Since the number and structure of CASs vary between samples, we constructed a Transformer-based semantic aggregator to integrate their semantic features and generate a global subtree embedding, which is aligned in dimension with the source code embedding. To further enhance semantic alignment, VulTrLM incorporates the vulnerability vector into the code representation process to well capture vulnerability patterns.

Finally, the fused representation is fed into a multilayer perceptron classifier to predict the presence of vulnerabilities.

To evaluate the effectiveness of VulTrLM, we conducted extensive experiments on three benchmark datasets: FFMpeg+Qemu (Zhou et al. 2019), Reveal (Chakraborty et al. 2021), and SVulD (Ni et al. 2023b). These datasets encompass a diverse range of C/C++ code snippets and vulnerability types. Our experimental results demonstrate that VulTrLM outperforms 12 state-of-the-art vulnerability detection methods, achieving F1 score improvements of 1.87%, 4.82%, and 12.46% on the respective datasets, surpassing the best baseline models. The ablation studies further validate the effectiveness of VulTrLM's core designs. In summary, this paper makes the following contributions:

- We propose VulTrLM, a novel vulnerability detection framework that effectively captures vulnerability patterns by enhancing code representations through the semantic aggregation of commented abstract subtrees.
- We propose the commented abstract subtree, a novel code representation that improves the semantic understanding of source code for pre-trained models. CASs are constructed via AST decomposition and enhanced with LLM-assisted natural language comments. To the best of our knowledge, it is the first of its kind in pre-trained models-based vulnerability detection tasks.
- We conduct extensive experiments to evaluate the performance of VulTrLM on three popular datasets, namely FFMpeg+QEMU, Reveal, and SVulD. Experimental results confirm the effectiveness of VulTrLM in vulnerability detection, with VulTrLM surpassing existing state-of-the-art methods in terms of F1 score.

The remainder of this paper is structured as follows. Section 2 surveys related work and provides the necessary background on vulnerability detection. Section 3 motivates our approach by demonstrating the limitations of pre-trained models on a representative real-world code example. Section 4 introduces the proposed framework, VulTrLM, detailing its design and components. In Section 5, we conduct a comprehensive evaluation of VulTrLM, comparing its performance against state-of-the-art baselines. Section 6 presents a comprehensive discussion of VulTrLM, including its applicability, limitations, and potential threats to validity. Finally, Section 7 concludes this paper and discusses future work.

2 Background and Related Work

Our approach integrates techniques from program analysis, deep learning, pre-trained models, and large language models. This section surveys the relevant background and reviews related work to our study.

2.1 Program analysis Methods

Program analysis techniques had a positive impact on vulnerability detection. Many of these techniques depend on simple pattern-based detection, which identifies security issues by locating predefined rules, as seen in tools like Flawfinder and PCA (Li et al. 2020). As techniques have progressed, more advanced analysis solutions have become available. These

include commercial solutions such as Checkmarx³ and CodeSonar,⁴ as well as open-source platforms like SonarQube⁵ and CodeQL.⁶ Even so, these rule-based methods are subject to manually defined rules by security experts (Kang et al. 2022), posing a barrier to covering all types of vulnerabilities comprehensively. To reduce the dependence on manual effort, recent work like IRIS (Li et al. 2025) has introduced the use of LLMs to infer taint specifications and perform context-aware analysis. This inspired our integration of LLMs with program analysis to enhance vulnerability detection. However, solely using program analysis methods results in high false positive rates when applied to complex real-world code.

2.2 Deep Learning Methods

In contrast to conventional program analysis techniques, DL-based approaches (Pornprasit and Tantithamthavorn 2022) are particularly effective at automatically extracting vulnerability patterns. These methods typically involve transforming source code into a representation compatible with machine learning, which is then fed into deep neural networks for feature extraction and classification. Broadly, these techniques fall into two categories: token-based methods and graph-based methods.

Token-based Methods These kinds of methods (Alon et al. 2018; Nguyen et al. 2021) preprocess source code through slicing and normalization, converting it into sequences of tokens for neural network learning. For instance, VulDeePecker (Li et al. 2018) feeds program slices into a BiLSTM model to learn vulnerability patterns, while SySeVR (Li et al. 2021b) enhances this by incorporating lexical analysis for more comprehensive slice extraction. VulDeeLocator (Li et al. 2022) further advances this line by leveraging intermediate representations and feeding them into a bidirectional recurrent neural network for fine-grained vulnerability detection and localization. However, token-based methods often struggle to capture the structural aspects of code, which can lead to suboptimal detection performance.

Graph-based Methods In recent years, graph-based code representations have shown significant promise in vulnerability detection. Through program analysis, source code is transformed into graph structures such as ASTs (Neamtiu et al. 2005), CFGs (Huo et al. 2020), PDGs (Xiao et al. 2024), or Code Property Graphs (CPGs) (Yamaguchi et al. 2014) and applies GNNs to learn meaningful representations. As illustrated in Fig. 1, an AST organizes source code into a hierarchical tree structure where nodes represent syntactic elements such as expressions, statements, and operators. CFGs, PDGs, and CPGs build on ASTs by incorporating control and data dependency information. In VulTrLM, we adopt ASTs as our primary representation to effectively preserve the structured semantics of source code.

Existing representative graph-based methods mainly focus on the overall structure of code, such as Devign (Zhou et al. 2019), Reveal (Chakraborty et al. 2021), and FUNDED

³<https://checkmarx.com/>

⁴<https://codesecure.com/our-products/codesonar/>

⁵<https://www.sonarsource.com/products/sonarqube/>

⁶<https://codeql.github.com/>

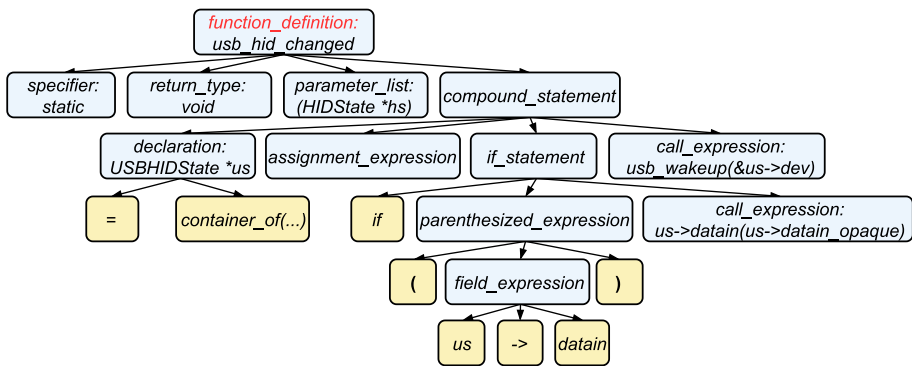


Fig. 1 Illustrative example of an AST structure

(Wang et al. 2020). However, these methods often fall short in capturing semantic information. To address this limitation, some researchers have explored ways to incorporate semantic context into graph representations to improve model performance. For example, MAGNET (Wen et al. 2024b) constructs a multi-granularity graph representation based on metapaths and employs a hierarchical attention GNN for vulnerability detection. MGVD (Qiu et al. 2024) embeds AST information within a PDG to form a three-dimensional feature representation that fuses structural and semantic information using a CNN.

However, graph construction often comes with computational overhead. Moreover, DL-based approaches often struggle in real-world scenarios due to the scarcity of high-quality labeled data.

2.3 Pre-trained Models

To address the limitations of DL-based approaches in vulnerability detection, another alternative approach is to leverage pre-trained language models as backbone architectures. Pre-trained models from natural language processing have been applied to software engineering tasks due to the syntactic and semantic similarities between programming languages and natural language. Pre-trained models learn from large-scale code corpora through self-supervised learning and demonstrate consistent performance improvements as model and data scales grow (Wei et al. 2022). These models can be further fine-tuned to support diverse downstream tasks such as vulnerability detection (Zhang et al. 2023a; Hanif and Maffei 2022).

Models Most pre-trained models in software engineering inherit architectural paradigms originally designed for natural language processing. For instance, CodeT5 (Wang et al. 2021) follows the text-to-text paradigm of T5 (Raffel et al. 2020), and introduces identifier-related tagging and prediction mechanisms, enabling bidirectional generation of code and comments. CodeBERT (Feng et al. 2020) employs a dual-modal framework that jointly processes natural language and programming language inputs, learning cross-modal representations through a replaced token detection objective. In addition, several studies have proposed enhancements to better capture code semantics. TreeBERT (Jiang et al. 2021) flattens ASTs for input into a Transformer-based encoder-decoder model and incorporates node

path information to enhance syntactic representation. GraphCodeBERT (Guo et al. 2020) integrates data flow dependencies into code representations, while UniXcoder (Guo et al. 2022), built on the RoBERTa architecture, unifies code tokens, AST structures, and comments. Accordingly, we adopt UniXcoder as the foundational backbone of VulTrLM for its ability to effectively model both semantic and structural information.

Application In the context of vulnerability detection, pre-trained models demonstrate efficient advantages by leveraging general knowledge. For example, VulBERTa (Hanif and Maffei 2022) employs byte pair encoding and predefined code tokens, and shows strong detection performance when fine-tuned on the GitHub and Draper datasets (Russell et al. 2018). Similarly, SecureFalcon (Ferrag et al. 2025) employs Falcon-40B as its pre-trained base model, which is subsequently fine-tuned on a domain-specific dataset to enhance the accuracy of vulnerability predictions. In addition, EPVD (Zhang et al. 2023a) introduces an execution path selection algorithm to learn path-specific representations, while SVulD (Ni et al. 2023b) enhances semantic discrimination through contrastive pairing. SCALE (Wen et al. 2024a) innovatively constructs structured natural language comment trees to help pre-trained models capture deep semantic features of code. Vul-LMGNNs (Liu et al. 2025a) integrates pre-trained CodeLMs with GNNs via online knowledge distillation to strengthen semantic information.

However, most existing pre-trained model-based approaches typically treat code as a sequence of tokens. It makes it difficult to capture complex logic and multiple execution paths, which limits their effectiveness in vulnerability detection. VulTrLM is designed to address these issues by enabling pre-trained models to more effectively detect vulnerabilities.

2.4 Large Language Models

In recent years, the application of LLMs in software engineering has garnered increasing attention (Zhu et al. 2025; Zhou et al. 2025). The evolution from GPT-3 to Codex (Chen et al. 2021b) and further to GPT-4⁷ has remarkably advanced the capabilities of LLMs in processing and understanding programming languages. The latest generation of code-specific LLMs, such as CodeLLaMA by Meta (Roziere et al. 2023), Qwen2.5-Coder by Alibaba Cloud (Hui et al. 2024), and DeepSeek-Coder by DeepSeek (Guo et al. 2024; Deng et al. 2025a), have been trained on vast, high-quality code repositories and now play a pivotal role in tasks such as code comprehension, generation, and documentation.

However, the substantial progress LLMs have achieved in code comprehension does not yet extend to vulnerability detection, mainly due to a lack of domain-specific security knowledge. This deficiency particularly impairs their ability to analyze code containing numerous undefined identifiers.

Overall, current research in vulnerability detection has evolved from rule-based analysis to deep learning models and further to pre-trained and large language models. These approaches differ in how they represent and process code. Some focus on syntax and structure, others on semantics and contextual reasoning. However, each type has its limitations when used alone. Table 1 summarizes key characteristics of these representative methods.

⁷<https://chatgpt.com/>

Table 1 Distinguishing characteristics of VulTrLM compared with existing methods

Category	Method	Initial Representation	Model Inputs	Model
Token-based	VulDecPecker (Li et al. 2018)	Code	Code Gadgets	BLSTM
	SySeVR (Li et al. 2021b)	PDG	SeVCs	BiGRU
	VulDecLocator (Li et al. 2022)	PDG	iSeVCs	BiRNN+Attention
Graph-based	Devign (Zhou et al. 2019)	CPG	CPG	GGNN
	Reveal (Chakraborty et al. 2021)	CPG	CPG	GGNN
	FUNDED (Wang et al. 2020)	Code snippets	Augmented AST	GGNN
	IVDetect (Li et al. 2021a)	PDG	PDG	FA-GCN
	MGVD (Qiu et al. 2024)	PDG+AST	3D Feature Matrix	CNN
LLM Pretrained	ChatGPT	Code	Code+Prompt	GPT
	VulBERTa (Hanif and Maffei 2022)	Code	Code	RoBERTa
	EPVD (Zhang et al. 2023a)	CFG+AST	CFG+AST	CodeBERT+CNN
	SVulD (Ni et al. 2023b)	AST	AST	UniXcoder
	SCALE (Wen et al. 2024a)	AST	SCT	UniXcoder
	VulTrLM	AST	CAS	UniXcoder+Transformer

In this paper, VulTrLM aims to integrate structural representations, semantic modeling, and LLM capabilities into a unified framework to enhance vulnerability detection performance.

3 Motivation

To further illustrate the two limitations of pre-trained models discussed in Section 1 to vulnerability detection, we analyze a real-world code example. Figure 2 presents a `NULL` pointer dereference vulnerability in the `decode_frame()` function from the FFmpeg project.⁸ This case study reveals two critical shortcomings of pre-trained models:

(1) Pre-trained models may fail to infer the semantics of statements with complex logic. These models rely heavily on the statistical patterns (Prystawski et al. 2023) of tokens like symbols, keywords, and operators. Because of this, they tend to focus on local token relationships while missing the overall logic of a statement. Take the example in Fig. 2. Lines 3–4 define two variables, `has_tile_bits` and `has_strip_bits`, using multiple operators (`||` and `->`). These flags are used to determine if the input contains *tiled-* or *strip-based* image data. Ideally, the model should recognize that both being true triggers a warning and early exit (line 5), implying a mutual exclusivity constraint. However, if the model treats each operator and variable as isolated elements, it may fail to detect this semantic rule. Similarly, the condition on line 8 appears to be a normal safety check for the model. Since pre-trained models are mainly trained on surface-level

⁸<https://github.com/FFmpeg/FFmpeg/commit/292e41ce650a7b5ca5de4ae87ff0d6a90d9fc97>


```

1  static int decode_frame(AVCodecContext *avctx,
        void *data, int *got_frame, AVPacket *avpkt){
2      ...
3      has_tile_bits = s->is_tiled || s->tile_byte_counts_offset || ...;
4      has_strip_bits = s->strippos || s->strips || ...;
5      if (has_tile_bits && has_strip_bits) {
6 -         av_log(avctx, AV_LOG_WARNING, "...");
+         int tiled_dng = s->is_tiled && is_dng;
+         av_log(avctx, tiled_dng ? AV_LOG_WARNING : AV_LOG_ERROR, "...");
+         if (!tiled_dng)
+             return AVERROR_INVALIDDATA;}
7      ...
8 -     if (!s->is_tiled) {
+     if (!s->is_tiled || has_strip_bits) {
9         if (s->strips == 1 && !s->stripsize) {...}
10        if (s->stripsizesoff) {
11            if (s->stripsizesoff >= (unsigned)avpkt->size)
12                return AVERROR_INVALIDDATA;
13            ...}
14        if (s->strippos) {
15            if (s->strippos >= (unsigned)avpkt->size)
16                return AVERROR_INVALIDDATA;
17            ...}
18        if (s->rps <= 0 || s->rps % s->subsampling[1]) {
19            ...;
20            return AVERROR_INVALIDDATA;}
21        ...
22    }

```

Fig. 2 The Ffmpeg code example for CVE-2020-36138 illustrates a vulnerability involving complex code logic. Lines in red represent the code before the fix, while lines in green show the corrected version

patterns, they may treat this as a typical error-handling condition and overlook the fact that when *tiled* is true, strip-related fields still require validation. This leads to a missed vulnerability where unsafe *strip* data could bypass checks entirely.

(2) Pre-trained models may struggle to understand execution paths. These models tend to treat source code as flat token sequences, ignoring branching and non-linear execution paths such as those dictated by *if* and *return* statements. In the example from Fig. 2, the use of *if* (e.g., lines 10, 14, and 18) and *return* directly contributes to a vulnerability. Some *return* occur only when multiple *if* conditions are satisfied. It is difficult for pre-trained models to predict potential vulnerabilities without accurately modeling such execution paths. Based on these observations, we adopt AST as the foundational source code representation for pre-trained models. In contrast to raw code, AST explicitly

captures syntactic structures and control dependencies in a hierarchical manner, helping models better understand the semantic flow of execution. As discussed in Section 1, raw code often lacks explicit indicators of operator precedence and contextual logic. This absence would lead to inaccurate model inference, especially when processing nested logic or multi-branch conditions. Additionally, directly inputting complete functions into a pre-trained model may obscure vulnerability patterns in complex code fragments. Intuitively, we can decompose each function's AST into multiple subtrees, where each subtree corresponds to a specific statement or logical unit. Each subtree is paired with high-quality comments generated by LLMs, providing extra contextual information to help pre-trained models better understand code semantics. Moreover, AST naturally preserves control flow paths, making execution flow more explicit. For example, `if` nodes indicate branching logic, while `for` nodes reveal loop structures. To further capture control dependencies in execution order, subtrees are arranged according to the original statement order, and a parent-child mask matrix is introduced to model structural and control relationships among subtrees explicitly.

4 Proposed Framework

In this section, we provide a detailed overview of the overall architecture of VulTrLM. As illustrated in Fig. 3, VulTrLM comprises five core modules: (1) AST decomposition, (2) LLM-assisted CAS construction, (3) Unified representation, (4) Feature fusion, and (5) Vulnerability detection. These modules process the target source code snippets in both the training and testing phases. The subsequent sections offer a detailed description of each module.

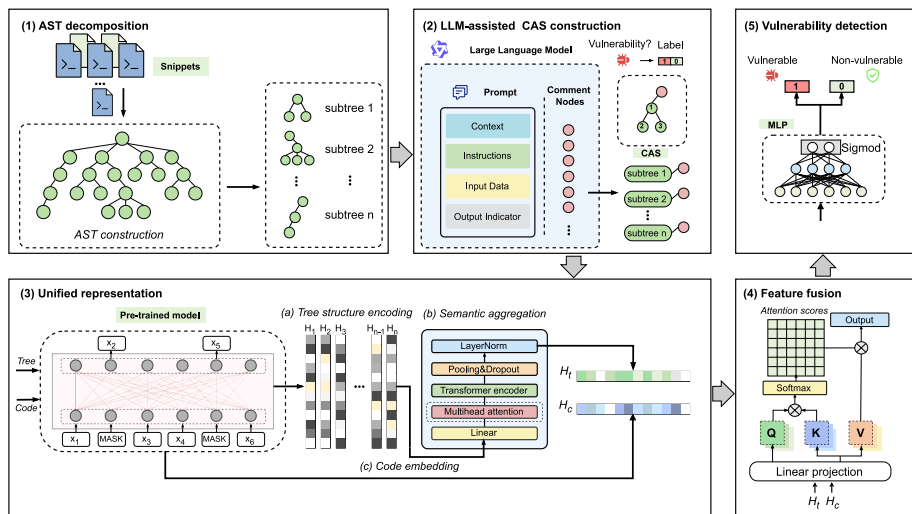


Fig. 3 The architecture of VulTrLM

4.1 AST Decomposition

In this stage, we focus on decomposing the AST of each code sample. We used Tree-sitter,⁹ a lightweight parsing library for C/C++, to construct the AST of each code sample. As a fundamental representation of code, ASTs abstract away syntax details while preserving program structure, making them valuable for many code-related tasks (Jiang et al. 2021; Wang and Li 2021; Bai et al. 2021). Even so, it can hinder model comprehension and raise computational costs as the complexity of ASTs increases. To address this, researchers have explored various ways to simplify ASTs for downstream tasks, such as flattening them into token sequences (Zhang et al. 2020; Svyatkovskiy et al. 2019) or representing them as sets of paths (Kim et al. 2021; Jiang et al. 2021). However, both methods risk losing essential structural and semantic information. Another approach more relevant to our work is to use subtree-based methods, which divide the AST into multiple subtrees. For instance, ASTNN (Zhang et al. 2019) splits ASTs at the statement level, and InferCode (Bui et al. 2021) further decomposes expressions and small elements to enhance granularity. Besides, TrVD (Tian et al. 2024) prunes ASTs using fixed-depth thresholds to retain essential structural cues.

However, both ASTNN and TrVD adopt a parallel arrangement of subtrees based on their code order, ignoring the parent-child relationships between subtrees, similar to flattening code indentation. For example, as discussed in Section 3, when two nested *if* statements are split into separate subtrees, their hierarchical structure is lost. It disrupts the original control flow and weakens the model's ability to capture the contextual semantics necessary for identifying vulnerabilities.

To solve this problem, VulTrLM extends this line of work (Tian et al. 2024) by proposing an improved AST decomposition algorithm that preserves parent-child relationships, helping maintain code hierarchy and control flow.

Algorithm 1 outlines the procedure of our subtree decomposition. Specifically, given an AST, the decomposer first traverses and records each node, including statements, expressions, and identifiers, while identifying control flow nodes (e.g., *if*, *for*, *while*) as potential decomposition points. For each candidate node, the depth d and node count n of the corresponding subtree are computed. The determination to initiate decomposition is based on predefined thresholds ($d_{max} = 8$, $n_{max} = 40$). If the conditions $d \leq d_{max}$ and $n \leq n_{max}$ are satisfied, the node and its subtree are extracted as a semantic unit and added to the results set. To preserve structural relationships, each subtree is assigned a unique *parent_id* to indicate its direct parent subtree. Additionally, the ordering of subtrees in the results set follows their original code sequence, maintaining execution logic. This indexing mechanism not only facilitates independent subtree processing but also enables efficient reconstruction of

⁹<https://tree-sitter.github.io/tree-sitter/>

the control flow structure via *parent_id*, providing hierarchical contextual information for subsequent semantic encoding.

Algorithm 1 AST decomposition.

Input:

T_{ast} : Input AST.
 d_{max} : the maximum depth of a subtree.
 n_{max} : the maximum number of nodes in a subtree.

Output:

$S = \{(id, parent\ id, subtree)\}$: Decomposed subtree set.

```

1: function DECOMPOSE( $T_{ast}, d_{max}, n_{max}$ )
2:    $S \leftarrow \emptyset$ ;
3:    $id \leftarrow 0$ ;
4:   for all  $node \in \text{Traverse}(T_{ast})$  do
5:     if ISStmt( $node$ ) then
6:        $d \leftarrow \text{trDepth}(node)$ ;
7:        $n \leftarrow \text{trSize}(node)$ ;
8:       if  $d > d_{max} \vee n > n_{max}$  then
9:          $split\_nodes \leftarrow \text{do\_split}(node)$ ;
10:        for all  $s \in split\_nodes$  do
11:           $S \leftarrow S \cup (\text{getID}(), node.id, s)$ ;
12:        end for
13:      else
14:         $S \leftarrow S \cup (\text{getID}(), \text{getParentID}(node), node)$ ;
15:      end if
16:    else
17:       $S \leftarrow S \cup (\text{getID}(), \text{getParentID}(node), node)$ ;
18:    end if
19:  end for
20:  return  $S$ 
21: end function
22: function ISSTMT( $node$ ) return
    $node.type \in \{\text{'FuncDef'}, \text{'If'}, \text{'For'}, \text{'DoWhile'}, \text{'While'}, \text{'Switch'}, \text{'Try'}\}$ ;
23: end function

```

Figure 4 illustrates an example to further clarify our AST decomposition algorithm. More specifically, given the code snippet shown in Fig. 4(a) as input, our algorithm decomposes its AST into 12 subtrees. Each subtree is assigned a unique ID and annotated with its parent-child relationship, maintaining the original execution sequence. Notably, the *if* statement on lines 5–6 is retained as a single subtree since it falls within the threshold, whereas the *for* loop on lines 7–8 exceeds the limits and is further split into smaller subtrees.

4.2 LLM-assisted CAS Construction

This section aims to integrate the syntactic structure and functional semantics of each subtree by constructing CASs. Instead of replacing function and variable names with generic placeholders, we retain the original identifiers, as they often carry meaningful semantics that help understand code behavior and improve the quality of generated descriptions. To enrich these structural representations, we leverage an LLM to generate high-quality natural

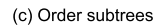


Fig. 4 This example illustrates the results of our AST decomposition applied to a code snippet. (a) presents the input code, while (b) shows how a complete subtree is decomposed into multiple smaller subtrees, with blue squares indicating their roots. The dashed blue boxes indicate parts of the AST that were left intact because they did not exceed the preset thresholds for depth or number of nodes. Finally, (c) displays the decomposed structure where each subtree is assigned a unique ID along with its parent-child relationships

language comments for each syntactic subtree. By doing so, we combine the “how” (i.e., AST) with the “why” (i.e., comment) to form a comprehensive understanding of the code.

To support comment generation for each subtree, we adopt Qwen2.5-Coder-7B-Instruct as the underlying LLM, as it is specifically optimized for code understanding and generation tasks. Compared to proprietary large models (e.g., GPT-4), it offers a better trade-off between inference efficiency and output quality, while also providing more flexibility for deployment and reduced computational cost. Furthermore, Qwen2.5-Coder-7B-Instruct can directly parse and understand syntactic subtrees, enabling it to identify key elements such as control flow structures and generate informative, targeted comments accordingly.

To ensure the quality and consistency of generated comments, we design a concise and effective prompt template as shown in Fig. 5. This prompt guides the model to produce

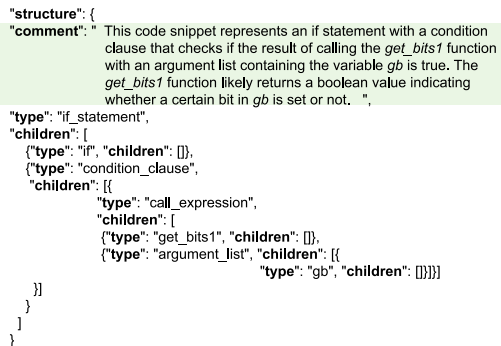


Fig. 5 Prompts for LLMs use simplified structures to focus on functionality. Full subtree details are left out for clarity

clear and accurate descriptions of functionality, focusing on the intended semantics behind the code rather than providing overly verbose or purely syntactic explanations. We set the temperature parameter to 0 to eliminate generation randomness, ensuring deterministic and reliable outputs. After generation, a post-processing step is applied to remove irrelevant formatting symbols and special characters such as asterisks (*) and semicolons (;).

In this final stage of the module, the cleaned comment is attached as an additional node to the root of each subtree, forming the CAS representation that fuses syntactic structure with semantic description. This enriched representation provides valuable contextual information and semantic background for downstream tasks.

4.3 Unified Representation

Most existing approaches rely on transformed representations of code, often overlooking the semantic information embedded in the source code. To fully exploit the latent correlations between source code and CASs, we propose a unified encoding strategy that maps both modalities into a shared semantic space.

Specifically, we adopt UniXcoder, a unified cross-modal (i.e., code, comment, and AST) pre-trained model. In VulTrLM, the source code and CASs are both encoded using UniXcoder. The source code embeddings are used directly, while the CAS embeddings are passed through a semantic aggregator to obtain a global representation for each subtree. The remainder of this section details the three components that implement the above pipeline: Code embedding (Section 4.3.1), Tree structure encoding (Section 4.3.2), and Semantic aggregation (Section 4.3.3). In the end, it results in two embeddings with the same dimensions to facilitate semantic alignment.

4.3.1 Code Embedding

To obtain rich semantic representations of source code, we use pre-trained models such as UniXcoder (Guo et al. 2022) to convert source code into dense vector embeddings. Unlike traditional methods (e.g., Word2Vec, BPE, Sent2Vec) that assign fixed vectors to tokens, UniXcoder generates context-aware embeddings. It means the same token can have different representations depending on its usage in different code contexts.

We input the source code into UniXcoder and extract the token embeddings from the final layer. These embeddings capture both the token values (e.g., identifiers, keywords) and their syntactic context. During training, we fine-tune only the final layer of UniXcoder while keeping the other layers frozen. It can retain the general programming knowledge learned during pretraining and allows the model to focus on the vulnerability detection task. Then, all embeddings are projected into a 128-dimensional space to enhance stability and reduce computation.

4.3.2 Tree Structure Encoding

Given a set of CASs $\mathcal{T} = \{T_1, T_2, \dots, T_m\}$, each subtree T_i obtained from AST decomposition is represented as a tuple (S_i, C_i) , where S_i denotes the syntactic structure (i.e., an AST fragment), and C_i is the natural language comment generated by the LLM.

To make CASs compatible with our encoder (the same pre-trained model used in Section 4.3.1), we define a mapping tree-specific transformation function F , as shown in Algorithm 2. This function performs a recursive pre-order traversal over the tree structure S_i , while incorporating the corresponding comment C_i as a special node. The transformation uses boundary markers to preserve the tree's structure and distinguish between node types and comment content.

The transformation process follows specific formatting rules for different node types, detailed as follows:

- For each comment C_i , we prepend and append boundary markers $\langle comment, left \rangle$ and $\langle comment, right \rangle$ to encapsulate the comment content.
- For leaf nodes in S_i , the algorithm directly outputs the node type without additional markers.
- For non-leaf nodes in S_i , we wrap the child information with boundary markers $\langle node_type, left \rangle$ and $\langle node_type, right \rangle$ to explicitly encode the hierarchical structure.

Algorithm 2 CAS sequence Converter $\mathcal{F}$.

Input: The root node $root$ of $T_i = (S_i, C_i)$.

Output: A flattened subtree sequence.

```

1: function  $\mathcal{F}(root)$ 
2:    $seq \leftarrow []$ ,
3:   if  $root.type = comment$  then
4:      $seq \leftarrow [\langle comment, left \rangle] \oplus root.text \oplus [\langle comment, right \rangle]$ ,
5:   else if  $root.isLeaf$  then
6:      $seq \leftarrow [root.type]$ ,
7:   else
8:      $seq \leftarrow [\langle root.type, left \rangle]$ ,
9:     for each  $child$  in  $root.children$  do
10:       $seq \leftarrow seq \oplus \mathcal{F}(child)$ ,
11:     end for
12:      $seq \leftarrow seq \oplus [\langle root.type, right \rangle]$ ,
13:   end if
14:   return  $seq$ 
15: end function

```

Following the transformation of the CASs into a structured representation, each subtree T_i is subsequently encoded into a fixed-dimensional vector representation $H_i \in \mathbb{R}^d$.

4.3.3 Semantic Aggregation

To further capture interactions among the CASs, we design a Transformer-based semantic feature aggregator. The aggregator explicitly considers the control-flow relationships between subtrees when computing attention, allowing it to assign attention weights in a structure-aware manner. Specifically, it enables each subtree to selectively attend to others that are structurally or semantically related, thereby capturing long-range dependencies across the entire set. To guide the attention computation, we construct a parent-child mask matrix $M \in \{-inf, 1\}^{m \times m}$, where m is the number of CASs. Notably, each entry $M_{i,j}$ indicates whether subtree i can attend to subtree j .

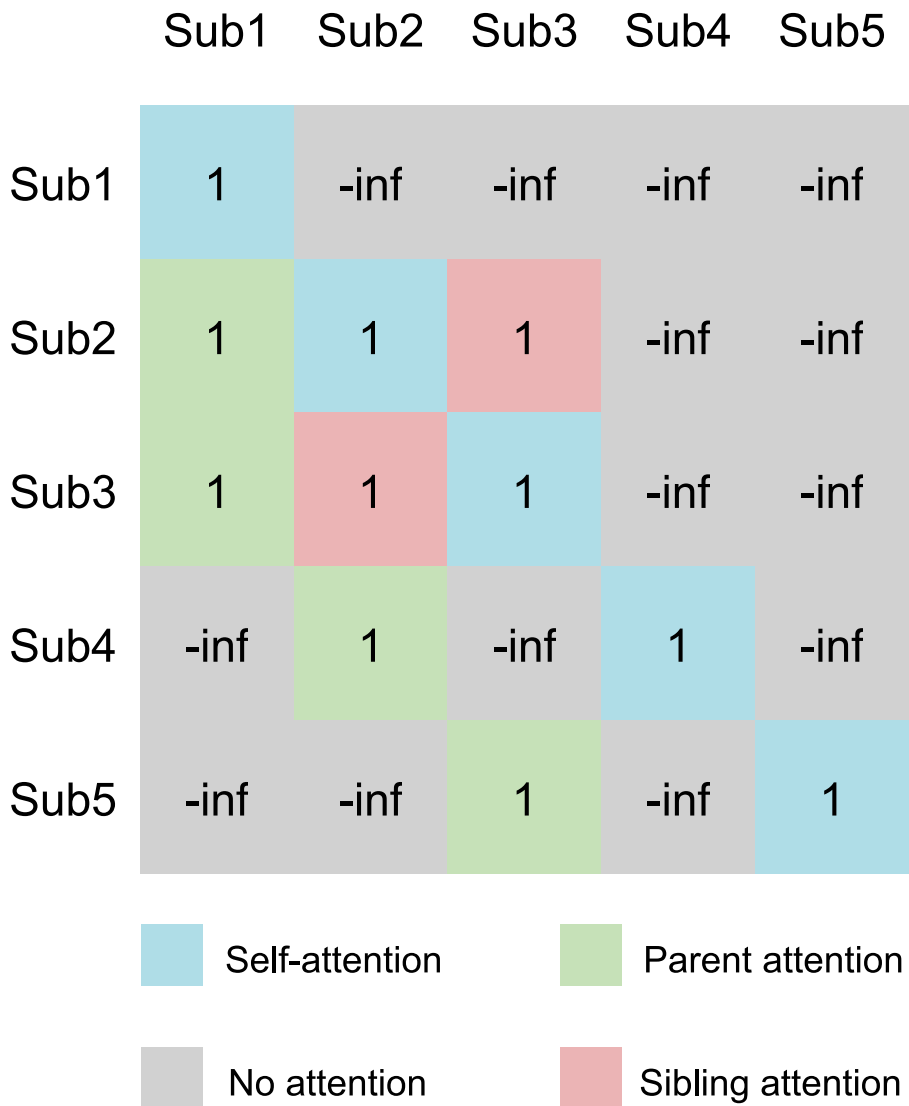


Fig. 6 Parent-child mask matrix

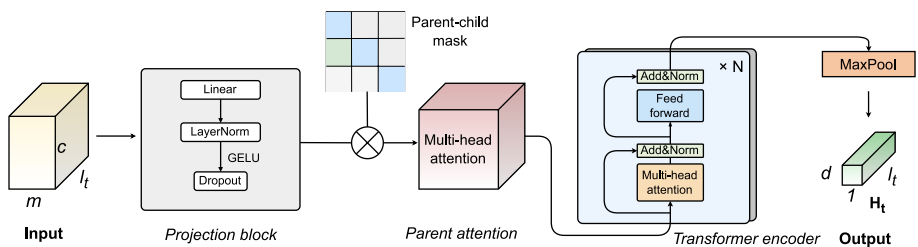


Fig. 7 Pipeline of the semantic aggregator

As illustrated in Fig. 6, the matrix is generated according to the following rules:

- **Self-attention:** Each subtree is allowed to attend to itself, i.e., $M_{i,i} = 1$, ensuring that local semantics are retained.
- **Parent attention:** If subtree i has subtree j as its parent or child, then $M_{i,j} = 1$, enabling semantic propagation from child to parent.
- **Sibling attention:** If subtrees i and j share the same parent, then $M_{i,j} = M_{j,i} = 1$, facilitating semantic interactions among sibling nodes.

Under this masking constraint, VulTrLM employs a multi-head attention mechanism to aggregate contextual information from structurally related subtrees, as detailed in Fig. 7. On top of this, we further stack four Transformer encoder layers to propagate semantic information throughout the entire tree, enabling progressive integration of global context. To derive a global representation from the subtree set $\{H_1, H_2, \dots, H_m\}$, we apply max pooling across the feature dimension as (1):

$$H_t = \max(H_1, H_2, \dots, H_m), \quad (1)$$

Our empirical dataset analysis reveals that while functions generate varying numbers of subtrees, over 96% produce no more than 64 subtrees. Consequently, we impose an upper bound of 64 on the number of subtrees to balance model complexity and training stability.

4.4 Feature Fusion

Unlike unimodal baselines that focus on a single source of information (Zhou et al. 2019; Zhang et al. 2024a; Guo et al. 2020), VulTrLM captures both high-level semantics and fine-grained structural cues. In this subsection, we aim to achieve semantic alignment between the source code and its corresponding subtrees. The source code is encoded into a sequential representation $H_c \in \mathbb{R}^{l_c \times d}$, while the global subtree representation $H_t \in \mathbb{R}^{l_t \times d}$ is obtained via the semantic aggregator. Here, l_c and l_t denote the lengths of the code and subtree sequences, respectively, and d is the hidden dimension size. To enable interaction between the two modalities (Zhang et al. 2025a), VulTrLM adopt a cross-attention (Chen et al. 2021a) mechanism, where the subtree embedding H_t serves as the query and the code embedding H_c acts as the key and value sources. These embeddings are first projected into the query, key, and value matrices through trainable linear transformations, as formulated in (2):

$$Q = H_t W_q, \quad K = H_c W_k, \quad V = H_c W_v, \quad (2)$$

where $W_q, W_k, W_v \in \mathbb{R}^{d \times d}$ are trainable projection parameters. Based on these matrices, the attention weights are computed using the standard scaled dot-product attention mechanism, as formulated in (3):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V, \quad (3)$$

where $K^\top$ denotes the transpose of K , and $\text{softmax}(\cdot)$ normalize the attention scores into a probability distribution.

Additionally, to capture diverse vulnerability patterns, we use multi-head cross-attention and concatenate the outputs to obtain the fused representation $H_f \in \mathbb{R}^{l_t \times d}$. Finally, a mean pooling operation is applied over the sequence to produce the unified embedding $H_g \in \mathbb{R}^d$.

4.5 Vulnerability Detection

The fused vector $H_g \in \mathbb{R}^d$ is fed into a multilayer perceptron (MLP) classifier, which serves as a discriminative model to predict vulnerability. Specifically, a fully connected layer followed by non-linear activation and dropout is used to generate a scalar probability, as formulated as (4):

$$y = \sigma (\tanh (\text{Dropout} (\text{Linear}(H_g)))) , \quad (4)$$

where σ denotes the Sigmoid activation function. Additionally, VulTrLM is trained by minimizing cross-entropy loss.

5 Experiment and Evaluation

In this section, we conduct comprehensive experiments to evaluate the effectiveness of VulTrLM and analyze the contributions of its core components. Specifically, we focus on the following research questions(RQs):

- RQ1: How effective is VulTrLM in the prediction of vulnerability at the function level?
- RQ2: How does the design of CAS influence the understanding of complex code structures?
- RQ3: How significant is the contribution of comments generated by LLMs to vulnerability detection?
- RQ4: Is VulTrLM consistently effective across different pre-trained models?
- RQ5: How do hyperparameters impact the performance of VulTrLM?

In RQ1, we aim to investigate the effectiveness of VulTrLM on the vulnerability detection task by comparing it with existing baseline methods (Section 5.2). In RQ2 and RQ3, we validate the impact of design choices on the performance of VulTrLM (Sections 5.3, 5.4). In RQ4, we examine whether our approach remains effective when integrated with pre-trained backbones other than UniXcoder (Section 5.5). In RQ5, we explore how the values of two key hyperparameters related to the model architecture affect the performance of VulTrLM (Section 5.6).

5.1 Experimental Setup

5.1.1 Datasets

We evaluate VulTrLM on three widely used vulnerability datasets to answer the above RQs. The details of these datasets are as follows.

FFMPeg+Qemu This dataset compiled by Zhou et al. (2019) includes samples from two widely used open-source projects involving FFmpeg and QEMU. It contains 22k function-level code samples, with 10k labeled as “vulnerable” and 12k as “non-vulnerable”, yielding a relatively balanced class distribution. Vulnerability labels were assigned by security experts who manually inspected the corresponding commits, and each label is tied to the original commit ID for full traceability.

Reveal This dataset (Chakraborty et al. 2021) is derived from historical vulnerability patches in the Linux Debian Kernel and Chromium projects. It contains 18K function samples with a notably unbalanced class distribution. Labels are based on reports from developers and users, reflecting the real-world distribution of software vulnerabilities. Every sample is linked to the patch ID and the issue ID for reproducibility.

SVulD This dataset, curated by Ni et al. (2023b) from the Big-Vul corpus, focuses on detecting subtle semantic differences in similar code snippets, covering 91 vulnerability types (e.g., buffer overflow, null pointer dereference). The labels are derived from comparisons of code before and after fixes, supplemented with CVE information.

All three datasets comprise real-world C/C++ vulnerability samples sourced from open-source projects and are widely used in prior research (Wen et al. 2024b; Zhang et al. 2023a; Wen et al. 2024a). We maintain their original distribution characteristics and randomly divide each dataset into non-overlapping training, validation, and test sets in an 8:1:1 ratio. The training set is used to learn model parameters, the validation set for hyperparameter tuning and model selection, and the test set solely for final performance evaluation. To ensure fair benchmarking, we retain a small number of code samples that cannot be successfully parsed into ASTs. These are represented as empty trees during processing and excluded from subtree decomposition. Detailed statistics for each dataset are provided in Table 2.

5.1.2 Evaluation Metrics

To comprehensively evaluate model performance, we adopt four widely used metrics. A confusion matrix is illustrated in Fig. 8, defined as follows:

Table 2 Statistics of vulnerability datasets used in the experiment

Dataset	#Vul	#Non-Vul	Ratio	Total
FFMPeg+Qemu	10,067	12,294	1:1.2	22,361
Reveal	1,664	16,505	1:9.9	18,169
SVulD	5,260	23,470	1:4.46	28,730

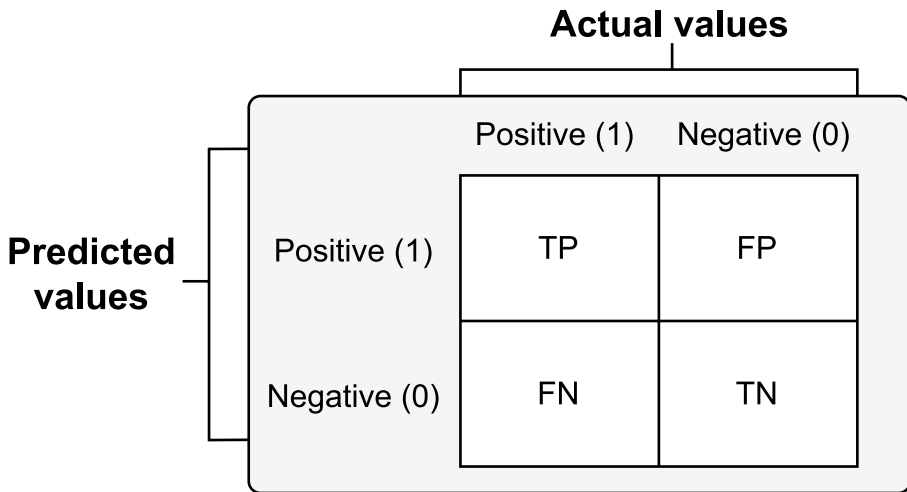


Fig. 8 Confusion matrix for abinary classification

- TP (True Positive): Number of correctly predicted vulnerable code samples.
- TN (True Negative): Number of correctly predicted non-vulnerable code samples.
- FP (False Positive): Number of non-vulnerable samples incorrectly predicted as vulnerable.
- FN (False Negative): Number of real vulnerabilities that the model failed to predict.

The evaluation metrics and their formulas are defined as follows:

- Accuracy (ACC): $ACC = \frac{TP+TN}{TP+TN+FP+FN}$. ACC measures the proportion of correctly classified samples.
- Precision (Pre): $Pre = \frac{TP}{TP+FP}$. Pre calculates the ratio of true vulnerable samples to all predicted vulnerable samples. Higher Pre indicates more reliable detection results.
- Recall (Rec): $Rec = \frac{TP}{TP+FN}$. Rec evaluates the model's ability to capture actual vulnerabilities. Higher Rec implies fewer false negatives.
- F1 score (F1): $F1 = \frac{2 \times Pre \times Rec}{Pre + Rec}$. F1 balances Pre and Rec through their harmonic mean.

5.1.3 Experimental Setting

All baseline models were implemented following their original publications. For the Devign model, due to missing hyperparameter details, we adopted the reproduction settings from Wen et al. (2023). To ensure fair comparison, all models used identical data splits and evaluation metrics.

VulTrLM is implemented in PyTorch for the vulnerability detection task. Input code was tokenized into sequences of up to 512 tokens. We used Tree-sitter to generate ASTs with maximum depth and node count thresholds set to 8 and 40, consistent with common practices in prior work (Tian et al. 2024).

For comment generation, we used the Qwen-2.5-Coder-7B-Instruct model from Huggingface, configured with a temperature of 0, a maximum generation length of 128 tokens, and a batch size of 8. LLM-based vulnerability detection methods were evaluated via official APIs from DeepSeek and Qwen.

VulTrLM consists of a 4-layer Transformer encoder with 128 hidden units and 4 attention heads. A dropout rate of 0.1 was applied to prevent overfitting. Training was conducted using the Adam optimizer with a learning rate of $2e-5$, fine-tuning from the UniXcoder. The batch size was set to 8. Hyperparameter impacts are further analyzed in Section 5.6. Training samples were shuffled at the start of each epoch.

All experiments were run on a server with an NVIDIA RTX 3090 GPU, Intel Xeon Silver 4214R CPU, and Ubuntu 20.04 LTS.

5.2 RQ1: How Effective is VulTrLM in the Prediction of Vulnerability at the Function Level?

In this experiment, we comprehensively evaluate VulTrLM by comparing it against a set of state-of-the-art vulnerability detection methods, including both non-LLM and LLM-based methods, using the previously introduced evaluation metrics in Section 5.1.2.

The baselines of non-LLM methods are as follows:

- **Devign** (Zhou et al. 2019), **Reveal** (Chakraborty et al. 2021), and **MAGNET** (Wen et al. 2024b) are graph-based methods. Devign augments CPG with natural-code-sequence edges and applies GGNN for vulnerability detection. Reveal also uses CPG but aggregates node embeddings via simple summation under GGNN. MAGNET adopts meta-path graphs instead.
- **CodeBERT** (Feng et al. 2020), **UniXcoder** (Guo et al. 2022), and **CodeT5** (Wang et al. 2021) are pre-trained models for general code understanding tasks. CodeBERT aligns code and natural language through joint pretraining. UniXcoder incorporates signals from code, comments, and ASTs in a unified framework. CodeT5 employs an encoder-decoder architecture and is pretrained on diverse code-related tasks.
- **VulBERTa** (Hanif and Maffeis 2022), **LineVul** (Fu and Tantithamthavorn 2022), and **EPVD** (Zhang et al. 2023a) are pre-trained model-based methods finetuned for vulnerability detection. VulBERTa is trained on large-scale C/C++ corpora. LineVul uses attention mechanisms for line-level detection and is adapted here for function-level analysis. EPVD encodes AST paths using pre-trained models.

For LLM-based methods, we choose the Qwen and DeepSeek model families, such as **Qwen2.5-7B-Instruct**, **Qwen2.5-Coder-7B-Instruct**, and **DeepSeek-v2.5**. These models balance deployment efficiency and computational cost while offering strong performance among existing large language models. To further improve LLM performance on vulnerability detection, we adopt a role-based prompt inspired by Fu et al. (2023).

Table 3 Comparison of evaluation results between VulTrLM and existing non-LLM vulnerability detection baselines on FFMpeg+Qemu, Reveal, and SVuID datasets

Method	FFMPeg+Qemu				Reveal				SVuID			
	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
DeSign	56.71	52.10	64.34	57.56	87.32	31.57	36.44	33.83	73.98	9.80	49.70	16.37
Reveal	60.92	55.30	70.20	61.86	81.65	31.68	60.83	41.66	82.40	12.80	39.60	19.34
MAGNET	63.28	56.27	80.15	66.12	91.60	42.86	61.68	50.57	86.99	12.68	56.84	20.73
CodeBert	61.24	56.86	64.70	60.53	89.20	46.33	54.55	50.10	78.25	14.63	66.16	23.96
UniXcoder*	61.60	57.57	62.39	59.89	88.25	38.89	31.82	35.00	81.25	12.57	57.60	24.14
UniXcoder	62.08	57.34	68.13	62.27	87.30	38.91	48.64	43.23	77.99	14.43	65.97	23.69
CodeT5	63.66	56.57	65.84	60.85	87.10	34.23	36.43	35.29	81.79	12.56	42.21	19.35
VulBERTa	62.59	62.83	50.76	56.15	84.12	34.46	62.25	44.36	80.11	16.45	67.78	26.47
LineVul	63.41	61.28	54.81	57.86	87.32	43.11	55.76	48.62	80.29	16.12	64.78	25.81
EPVD	63.03	59.32	62.15	60.70	88.89	38.68	60.93	47.32	76.75	14.22	69.61	23.62
VulTrLM	65.58	66.28	68.49	67.36	89.79	45.31	63.89	53.01	83.23	18.86	70.72	29.77

Bold values indicate the best results for each metric

UniXcoder* represents the variant without further fine-tuning

5.2.1 Comparison with non-LLM methods

We compared VulTrLM with non-LLM baseline methods, and the results are summarized in Table 3, with the best scores highlighted in bold. Specifically, VulTrLM outperformed baselines by 1.87% on the FFmpeg+QEMU dataset, 4.82% on the Reveal dataset, and 12.46% on the SVulD dataset. The findings indicate a notable effect of VulTrLM on improving the detection of code vulnerabilities.

We observe that graph-based supervised learning models (e.g., Devign and Reveal) tend to yield lower F1 score compared to pre-trained models. This performance gap can be attributed to the generalized knowledge representations that pre-trained models learn from large-scale code corpora during pretraining, which significantly enhances their ability to recognize subtle vulnerability patterns.

Notably, fine-tuned UniXcoder outperforms its counterpart, with F1 score gains of 3.97% and 29.22% on the FFmpeg+QEMU and Reveal datasets, respectively. This finding suggests that task-specific fine-tuning is crucial to adapt pre-trained models for effective vulnerability detection.

We find that VulTrLM outperforms the pre-trained model baselines in terms of F1 score by 8.18%, 5.81%, and 12.54% on the three datasets, respectively. This demonstrates that ATLAS is more effective in handling complex code structures and multi-branch control flows. Additionally, on the SVulD dataset, although all models show relatively low F1 score, VulTrLM achieves the best performance. This suggests that accurately capturing fine-grained semantic differences between vulnerable and fixed code remains a key challenge for future research.

5.2.2 Comparison with LLM-based Methods

We compared VulTrLM with several LLM-based approaches, including Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct, and DeepSeek-v2.5. Each model was prompted to internally analyze code snippets for potential vulnerabilities, with output restricted to a binary classification (0 for non-vulnerable, 1 for vulnerable). To maintain consistent evaluation, we used uniform prompts across all models and set the temperature to 0.01 to reduce stochastic variation.

As shown in Table 4, the highest F1 score achieved by the LLM is 58.35% on FFmpeg+QEMU, 18.83% on Reveal, and 17.29% on SVulD, below VulTrLM in each case. Similarly, LLM-based methods consistently underperform compared to non-LLM approaches across all datasets. One possible reason for this gap is that LLMs are primarily pretrained for code generation, rather than for learning vulnerability patterns. For example, on Reveal, DeepSeek-v2.5 attains a recall of 75.91% but a precision of only 10.75%, indicating many false positives. These results suggest that LLMs struggle with direct vulnerability detection due to limited domain-specific knowledge.

Table 4 Comparison of evaluation results between VulTrLM and existing LLM-based vulnerability detection baselines on FFMpeg+Qemu, Reveal, and SVulD datasets

Method	FFMPeg+Qemu			Reveal			SVulD					
	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
Qwen2.5-7B-Instruct	53.37	49.13	42.79	45.74	64.08	12.12	41.82	18.79	69.36	8.90	53.23	15.25
Qwen2.5-coder-7B-instruct	54.25	50.53	19.12	27.75	80.43	11.83	15.00	13.23	85.11	12.14	30.04	17.29
Deepseek-v2.5	57.80	46.06	79.60	58.35	64.93	10.75	75.91	18.83	69.14	6.34	92.02	11.86
VulTrLM	65.58	66.28	68.49	67.36	89.79	45.31	63.89	53.01	83.23	18.86	70.72	29.77

Bold values indicate the best results for each metric

Answer to RQ1: VulTrLM demonstrates significant advantages in function-level vulnerability detection. VulTrLM achieves a superior F1 score of 67.36%, 53.01%, and 29.77% on FFMPeg+Qemu, Reveal, and SVulD, respectively, surpassing the best baselines by 1.87%, 4.82%, and 12.46%.

5.3 RQ2: How does the Design of CAS Influence the Understanding of Complex Code Structures?

To answer RQ2, we conduct ablation studies by removing the CAS representation and the Parent-Child Mask (PCM) to investigate their respective contributions to model performance. We use only FFmpeg+QEMU for ablation due to its representativeness and resource limits. In addition, CAS and PCM are general modules independent of specific datasets. The experimental results are illustrated in Fig. 9.

5.3.1 Impact of Commented Abstract Subtree

To assess the effectiveness of CAS, we conducted an ablation study replacing it with the full AST. The Fig. 9 shows that this substitution led to a noticeable decline in performance on the FFmpeg+QEMU dataset, with the F1 score dropping from 67.36% to 64.28%, precision decreasing from 66.28% to 63.84%, and recall falling from 68.49% to 65.37%. These results underscore the pivotal role of CAS in enhancing the model's semantic comprehension of code, and highlight the superiority of subtree decomposition over full AST representation.

5.3.2 Impact of Parent-child Mask

We further investigate the impact of the PCM by removing all hierarchical relationships between subtrees and their parents, leaving the subtrees semantically isolated. As shown in Fig. 9, this modification resulted in a reduced F1 score from 67.36% to 64.58% and a slight decrease in accuracy from 65.58% to 65.32%, while recall experienced a more substantial drop from 68.49% to 61.56%. These findings suggest that PCM plays a critical role in feature aggregation by guiding the attention mechanism to focus on structurally related subtrees, thus preserving the hierarchical structure essential for the comprehension of execution sequences.

Answer to RQ2: The CAS significantly improves VulTrLM's performance, and its parent-child mask mechanism effectively captures contextual information between subtrees, enhancing the model's ability to detect vulnerability patterns.

5.4 RQ3: How Significant is the Contribution of Comments Generated by LLMs to Vulnerability Detection?

To investigate the effectiveness of LLM-generated comments within VulTrLM, we designed four experimental variants: a baseline utilizing raw subtree structures without comments, and three augmented variants incorporating comments generated by models with 0.5B, 1.5B, and 3B parameters, respectively.

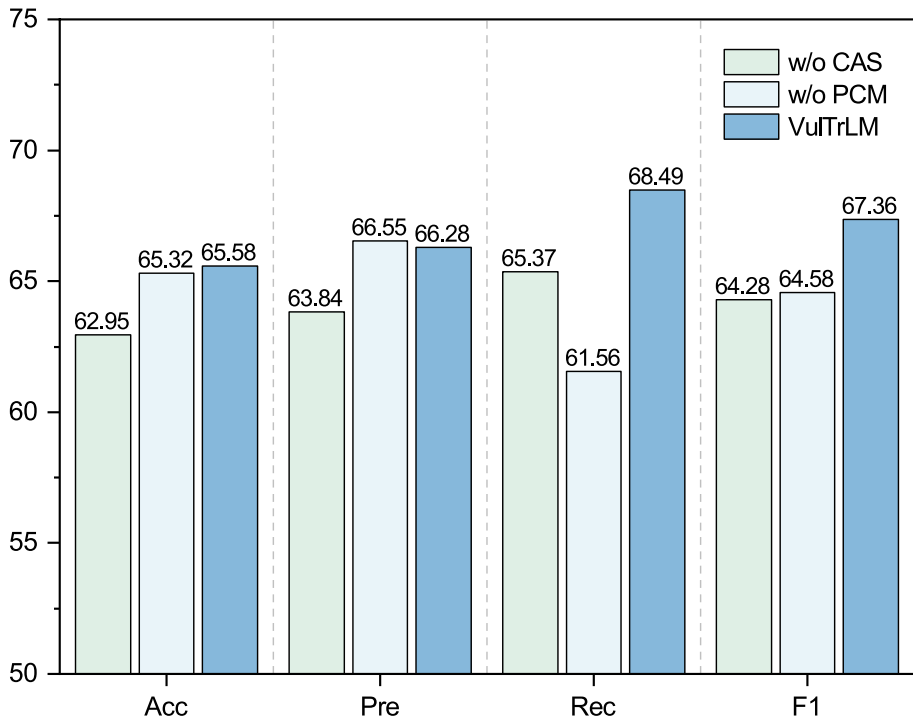


Fig. 9 Performance impact results between CAS and PCM on the FFmpeg+QEMU dataset

The results, summarized in Table 5, indicate that the baseline without generated comments outperformed the variants using comments from the 0.5B and 1.5B models. We hypothesize that smaller models generate lower-quality comments that fail to provide meaningful semantic supplementation, potentially introducing noise that impairs vulnerability pattern recognition. Further analysis reveals that smaller language models struggle to interpret prompts effectively, frequently generating code-like tokens instead of natural language comments that clearly describe code semantics.

Additionally, we observe diminishing returns in performance gains as model size increases under token constraints. While scaling from 1.5B to 3B parameters yields a 4.11% improvement in F1 score, further scaling to 7B results in only a 3.28% increase. This implies that, given the limited size of the subtree inputs, a 7B-parameter model is sufficient to capture the essential semantic information, and additional capacity offers minimal incremental benefit.

Table 5 Performance of VulTrLM variants with different LLM parameter sizes and a variant without comment on the FFmpeg+QEMU dataset

Method	Acc	Pre	Rec	F1
w/o comment	63.41	63.90	67.68	65.73
VulTrLM-0.5B	62.10	62.35	63.35	62.84
VulTrLM-1.5B	62.25	61.80	63.52	62.64
VulTrLM-3B	65.29	64.32	66.15	65.22
VulTrLM	65.58	66.28	68.49	67.36

All variants utilized the Qwen2.5-Coder-7B-Instruct model

Table 6 The experimental results of integrating CAS into existing pre-trained model-based approaches

Method	FFMPeg+Qemu						Reveal						SVulD					
	Acc	Pre	Rec	F1	Acc	F1	Acc	Pre	Rec	F1	Acc	F1	Acc	Pre	Rec	F1	Acc	F1
CodeBERT	64.25 (↑3.01%)	59.07 (↑2.21%)	72.11 (↑7.41%)	64.94 (↑4.41%)	89.78 (↑0.58%)	52.13 (↑2.03%)	80.06 (↑1.81%)	48.30 (↑1.97%)	56.62 (↑2.07%)	52.13 (↑2.03%)	80.06 (↑1.81%)	52.13 (↑2.03%)	80.06 (↑1.81%)	15.28 (↑0.65%)	62.74 (↓3.42%)	24.57 (↑0.61%)	80.06 (↑1.81%)	24.57 (↑0.61%)
CodeT5	66.18 (↑2.52%)	59.40 (↑2.83%)	72.35 (↑6.51%)	65.24 (↑4.39%)	89.49 (↑2.39%)	49.52 (↑13.09%)	76.83 (↓4.96%)	48.30 (↑14.07%)	50.81 (↑14.38%)	49.52 (↑13.09%)	76.83 (↓4.96%)	49.52 (↑13.09%)	76.83 (↓4.96%)	15.04 (↑2.48%)	62.93 (↑20.72%)	24.28 (↑4.93%)	76.83 (↓4.96%)	24.28 (↑4.93%)
UniXcoder	65.58 (↑3.50%)	66.28 (↑8.94%)	68.49 (↑0.36%)	67.36 (↑5.09%)	89.79 (↑2.49%)	53.01 (↑9.78%)	83.23 (↑5.24%)	45.31 (↑6.40%)	63.89 (↑15.25%)	53.01 (↑9.78%)	83.23 (↑5.24%)	53.01 (↑9.78%)	83.23 (↑5.24%)	18.86 (↑4.43%)	70.72 (↑4.75%)	29.77 (↑6.08%)	83.23 (↑5.24%)	29.77 (↑6.08%)

Answer to RQ3: Small models fail to produce meaningful comments, resulting in noisy outputs. In contrast, adequately large models generate high-quality comments that enhance subtree semantics, thereby improving the model's understanding of code.

5.5 RQ4: Is VulTrLM Consistently Effective Across Different Pre-trained Models?

In this research question, we investigate the impact of using different pre-trained backbones within VulTrLM and evaluate whether their effectiveness is comparable. To this end, we replaced UniXcoder with CodeBERT and CodeT5 to evaluate performance variations. The results are presented in Table 6.

The results show that integrating subtree structural information with natural language comments consistently enhances the performance of all pre-trained models. Across three different datasets, CodeBERT, CodeT5, and UniXcoder achieved average F1 score improvements of 2.35%, 7.47%, and 6.98%, respectively. Among them, UniXcoder with CASs delivered the best overall performance, likely due to its cross-modal pretraining on ASTs, which aligns well with the subtree decomposition strategy. Nevertheless, both CodeBERT and CodeT5 combined with CASs achieved a higher F1 score than most existing approaches. These results suggest that the subtree structure captures implicit code semantics, while the comments provide explicit logic, jointly improving the model's inference on complex code.

Answer to RQ4: The VulTrLM has been demonstrated to be effective across different pre-trained models. On three datasets, it improved the average F1 score of CodeBERT, CodeT5, and UniXcoder by 2.35%, 7.47%, and 6.98%, respectively.

5.6 RQ5: How do Hyperparameters Impact the Performance of VulTrLM?

In this research question, we explore the impact of two hyperparameters on the model's performance: the number of Transformer encoder layers and the number of attention heads. We vary the values of these hyperparameters and evaluate the model's performance across three datasets. The results are illustrated in Figs. 10 and 11, where blue indicates accuracy, yellow represents precision, dark purple denotes recall, and red means the F1 score.

5.6.1 Encoder Layer Numbers

We explore the effect of the encoder layer number. In Fig. 10, the results show a general trend of improvement followed by a decline as the number of layers increases. The optimal F1 score is achieved when the number of layers is set to 4, reaching 67.62% on FFmpeg+Qemu, 53.01% on Reveal, and 29.78% on SVulID, respectively. We suggest that increasing the depth of Transformer layers initially improves semantic aggregation (Deng et al. 2025c), but excessive layers may cause overfitting and hurt performance.

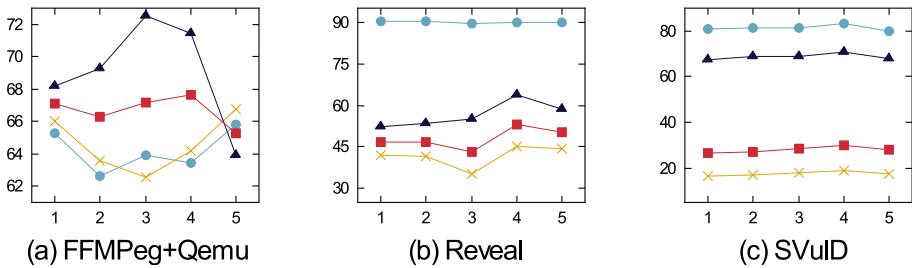


Fig. 10 The impact of encoder layer numbers on the model performance across all datasets

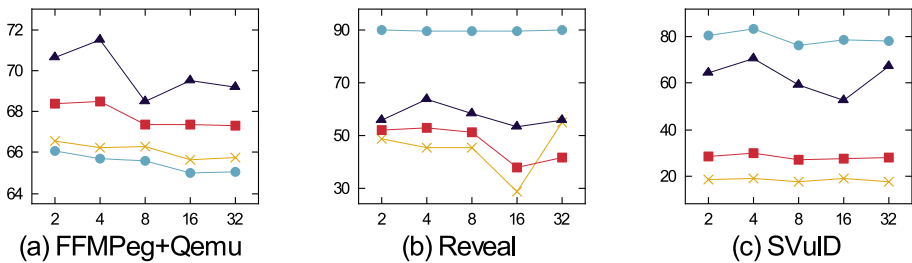


Fig. 11 The impact of attention head numbers on the model performance across all datasets

5.6.2 Attention Head Numbers

We analyze the effect of attention head numbers. We incrementally increased the number of heads from 2 to 32. The results in Fig. 11 indicate that VulTrLM achieves the highest F1 score on all datasets when using 4 attention heads. This pattern is consistent across datasets, and thus, we set the number of attention heads to 4 for all experiments. Overall, increasing the number of attention heads improves model performance up to a point, but performance begins to decline once the number exceeds 4.

Answer to RQ5: The number of Transformer encoder layers and attention heads significantly impacts VulTrLM's performance. Increasing the layers improves results up to 4 layers, but further additions cause overfitting. Similarly, 4 attention heads achieve the highest F1 score, while using more heads degrades performance.

6 Discussion

In this section, we present a comprehensive discussion of VulTrLM, including its applicability, limitations, and potential threats to validity.

6.1 Applicability

VulTrLM operates on ASTs to ensure its applicability. While alternative representations such as CFGs, PDGs, and XFGs (Cheng et al. 2021) can effectively capture execution

semantics, they often fail to parse code snippets with compilation defects due to their reliance on a complete compilation environment. In contrast, AST provides a structured representation that includes low-level statements and expressions and is easily compiled. During our AST parsing of three datasets, only a small number of samples generated empty subtrees due to missing critical code elements. The FFmpeg+Qemu dataset achieved 100% successful parsing, while Reveal and SVulD maintained low failure rates of 0.26% and 0.02%, respectively. Furthermore, the AST structure naturally encodes fine-grained syntactic details such as expressions and statements, supporting both syntactic and semantic reasoning when combined with large language models.

6.2 Limitation

Our approach still has the following limitations:

API Blind VulTrLM primarily focuses on function-level vulnerabilities. It is not well-suited for identifying issues that involve external API misuse. Addressing these complex vulnerability patterns will require incorporating function call graphs in future work. These types of vulnerabilities require a broader program-level view, which is beyond the current scope of our model.

Size Skew The varying complexity of code samples leads to uneven subtree structures. Complex or lengthy functions tend to produce many more subtrees than average, increasing computational cost and reducing semantic consistency. To manage this, we set a cap of 64 subtrees per code sample by applying a 95th percentile truncation strategy based on subtree distribution, which may cause slight information loss.

Prompt Drift Our approach relies on the quality of comments generated by the LLM. The accuracy and consistency of these comments are influenced by prompt design and model parameters, which in turn affect the overall detection performance.

6.3 Threats to Validity

To comprehensively assess the robustness of our study, we discuss potential threats to validity from both internal and external perspectives.

6.3.1 Internal Validity

Several factors may affect the internal validity of our experiments, including parameter settings, dataset quality, and hyperparameter tuning.

AST Parameters The AST decomposition parameters may influence results on complex code structures. We set the AST decomposition depth to 8 and the maximum number of nodes per subtree to 40 to balance context preservation and computational efficiency. However, for deeply nested code, these fixed thresholds may change the subtree structure. Although the current settings work well in general, they might need to be adjusted for specific code types.

Dataset Quality Dataset quality (Croft et al. 2023; Liu et al. 2023; Zhao et al. 2024) may also impact internal validity. Although FFmpeg+QEMU, Reveal, and SVulD are widely used and partially validated, they may still contain label noise due to automated or manual annotation errors. Despite this, they have shown strong utility in prior benchmarks, and our model is designed to tolerate a small amount of such noise.

Hyperparameter Tuning Due to hardware limitations, we did not perform extensive tuning of hyperparameters like learning rate and batch size. To ensure fairness and reproducibility, we kept the same settings across all experiments and followed the original configurations when running baselines. This reduces variability but may limit further performance gains.

6.3.2 External Validity

Our experiments with VulTrLM focus primarily on C/C++ code and do not yet cover other mainstream programming languages. Thus, its effectiveness on other programming languages remains to be verified. However, our approach relies on Tree-sitter for AST parsing, which currently supports over 40 widely used programming languages, including Java, Python, Go, and Rust. Furthermore, with the rapid advancement of large language models and the increasing availability of multilingual code corpora, models such as the Qwen series now support up to 92 programming languages. Therefore, in principle, VulTrLM can be applied to any programming language supported by the parser. Accordingly, we plan to extend our evaluation to additional languages in future work.

7 Conclusions

In this paper, we present VulTrLM, a vulnerability detection framework that enhances code semantics by decomposing ASTs and leveraging LLMs. We introduce CAS, a code representation that divides code into coherent and structurally complete subtrees. Each subtree is attached to a comment node generated by an LLM, and its features are extracted through a pre-trained model. We also design an aggregator to collect and integrate contextual information across subtrees. Compared to pre-trained model-based approaches, VulTrLM is more effective in handling complex code logic. Moreover, VulTrLM also shows general applicability when used with other pre-trained models. Experiments on the widely used datasets FFmpeg+QEMU, Reveal, and SVulD show that VulTrLM achieves F1 score improvements of 1.87%, 4.82%, and 12.46%, respectively. We also conduct comprehensive ablation studies to verify the effectiveness of our core designs. In future work, we plan to explore dynamic methods for evaluating subtree importance, enrich the structural representation of subtrees, and incorporate more powerful language models to further enhance semantic understanding.

Author Contributions All authors contributed to the conception and design of the study, implementation of experiments, and writing of the manuscript. All authors read and approved the final version of the manuscript.

Funding This work was supported in part by the National Natural Science Foundation of China (Grant Nos. 62272162, 62272150, 62472113, and 62172159), the Hunan Provincial Natural Science Foundation of China (Grant No. 2025JJ50398), the project of the Hunan Provincial Social Science Achievement Review Committee of China (Grant No. XSP25YBZ104), and the Soft Science Project of the National Intellectual Property Administration (Grant No. SS24-B-23).

Data Availability Statement The dataset used in this study is obtained from existing research, with sources properly cited in the manuscript and publicly accessible online. The remaining resources will be available on Zenodo.

Declarations

Ethical Approval No human participants or animals were involved in this study.

Informed consent Not applicable.

Consent for publication Not applicable.

Conflicts of Interest The authors declare no conflicts of interest related to this work.

Clinical Trial Number Not applicable.

References

- Alon U, Brody S, Levy O et al (2018) Code2seq: Generating sequences from structured representations of code. [arXiv:1808.01400](https://arxiv.org/abs/1808.01400)
- Bai J, Wang Y, Chen Y et al (2021) Syntax-bert: Improving pre-trained transformers with syntax trees. In: Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume, pp 3011–3020
- Bui ND, Yu Y, Jiang L (2021) Infercode: self-supervised learning of code representations by predicting subtrees. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, Madrid, Spain, pp 1186–1197
- Chakraborty S, Krishna R, Ding Y et al (2021) Deep learning based vulnerability detection: are we there yet? *IEEE Trans Software Eng* 48(9):3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- Chen CFR, Fan Q, Panda R (2021a) Crossvit: Cross-attention multi-scale vision transformer for image classification. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp 347–356
- Chen M, Tworek J, Jun H et al (2021b) Evaluating large language models trained on code. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374)
- Cheng X, Wang H, Hua J et al (2021) Deepwukong: statically detecting software vulnerabilities using deep graph neural network. *ACM Trans Softw Eng Methodol* 30(3):1–3. <https://doi.org/10.1145/3436877>
- Croft R, Babar MA, Kholoosi MM (2023) Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, Melbourne, Victoria, Australia, ICSE '23, pp 121–133
- CrowdStrike (2024) Crowdstrike 2024 threat hunting report. <https://www.securityweek.com/wp-content/uploads/2024/08/CrowdStrike-2024-Threat-Hunting-Report.pdf>
- CVE security vulnerability database (2025) Security vulnerabilities, exploits, references and more. <https://www.cvedetails.com/>
- Deng Q, Zuo Q, Li Z et al (2024) Blockchain-based reputation privacy preserving for quality-aware worker recruitment scheme in mcs. *IEEE/ACM Trans Networking* 32(6):5188–520. <https://doi.org/10.1109/TNET.2024.3453056>
- Deng Z, Ma W, Han Q et al (2025a) Exploring deepseek: a survey on advances, applications, challenges and future directions. *IEEE/CAA J Autom Sin* 12(5):872–89. <https://doi.org/10.1109/JAS.2025.125498>
- Deng Z, Sun R, Xue M et al (2025b) Hardening llm fine-tuning: from differentially private data selection to trustworthy model quantization. *IEEE Trans Inf Forensics Secur*. <https://doi.org/10.1109/TIFS.2025.3581103>

- Deng Z, Sun R, Xue M et al (2025c) Leakage-resilient and carbon-neutral aggregation featuring the federated ai-enabled critical infrastructure. *IEEE Trans Dependable Secure Comput*. <https://doi.org/10.1109/TDSC.2025.3532957>
- Feng Z, Guo D, Tang D et al (2020) CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the association for computational linguistics: EMNLP 2020. Association for Computational Linguistics, Online, pp 1536–1547
- Feng X, Zhu X, Han QL et al (2022) Detecting vulnerability on iot device firmware: a survey. *IEEE/CAA J Autom Sin* 10(1):25–41. <https://doi.org/10.1109/JAS.2022.105860>
- Ferrag MA, Battah A, Tihanyi N et al (2025) Securefalcon: are we there yet in automated software vulnerability detection with llms? *IEEE Trans Softw Eng*. <https://doi.org/10.1109/TSE.2025.3548168>
- Fu M, Tantithamthavorn C (2022) Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), Pittsburgh, Pennsylvania, MSR '22, pp 608–620
- Fu M, Tantithamthavorn CK, Nguyen V et al (2023) Chatgpt for vulnerability detection, classification, and repair: how far are we? In: 2023 30th Asia-Pacific Software Engineering Conference (APSEC). IEEE, Los Alamitos, CA, USA, pp 632–636
- Guo D, Ren S, Lu S et al (2020) Graphcodebert: Pre-training code representations with data flow. [arXiv:2009.08366](https://arxiv.org/abs/2009.08366)
- Guo D, Lu S, Duan N et al (2022) Unixcoder: Unified cross-modal pre-training for code representation. In: Proceedings of the 60th annual meeting of the association for computational linguistics (Volume 1: Long Papers), pp 7212–7225
- Guo D, Zhu Q, Yang D et al (2024) Deepseek-coder: When the large language model meets programming—the rise of code intelligence. [arXiv:2401.14196](https://arxiv.org/abs/2401.14196)
- Hanif H, Maffei S (2022) Vulberta: Simplified source code pre-training for vulnerability detection. In: 2022 International joint conference on neural networks (IJCNN). IEEE, pp 1–8
- Hui B, Yang J, Cui Z et al (2024) Qwen2.5-coder technical report. [arXiv:2409.12186](https://arxiv.org/abs/2409.12186)
- Huo X, Li M, Zhou ZH (2020) Control flow graph embedding based on multi-instance decomposition for bug localization. In: Proceedings of the AAAI conference on artificial intelligence, pp 4223–4230
- Jiang X, Zheng Z, Lyu C et al (2021) Treebert: A tree-based pre-trained model for programming language. In: Uncertainty in Artificial Intelligence, PMLR, pp 54–63
- Kang HJ, Aw KL, Lo D (2022) Detecting false alarms from automatic static analysis tools: How far are we? In: Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, Pennsylvania, ICSE '22, pp 698–709
- Kim S, Zhao J, Tian Y et al (2021) Code prediction by feeding trees to transformers. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, Madrid, Spain, ICSE '21, pp 150–162
- Li Z, Zou D, Xu S et al (2018) Vuldeepecker: A deep learning-based system for vulnerability detection. [arXiv:1801.01681](https://arxiv.org/abs/1801.01681)
- Li W, Cai H, Sui Y et al (2020) Pca: memory leak detection using partial call-path analysis. In: Proceedings of the 28th ACM Joint meeting on European software engineering conference and symposium on the foundations of software engineering, pp 1621–1625
- Li Y, Wang S, Nguyen TN (2021a) Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pp 292–303
- Li Z, Zou D, Xu S et al (2021b) Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans Dependable Secure Comput* 19(4):2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- Li Z, Zou D, Xu S et al (2022) Vuldeeloctor: a deep learning-based fine-grained vulnerability detector. *IEEE Trans Dependable Secure Comput* 19(04):2821–283. <https://doi.org/10.1109/TDSC.2021.3076142>
- Li Z, Dutta S, Naik M (2025) Iris: Llm-assisted static analysis for detecting security vulnerabilities. In: The thirteenth international conference on learning representations
- Liu Q, Peng Y, Xu Q et al (2023) Mars: enabling verifiable range-aggregate queries in multi-source environments. *IEEE Trans Dependable Secure Comput* 21(4):1994–201. <https://doi.org/10.1109/TDSC.2023.3299337>
- Liu R, Wang Y, Xu H et al (2025a) Vul-Imgnns: fusing language models and online-distilled graph neural networks for code vulnerability detection. *Inf Fusion* 115:10274. <https://doi.org/10.1016/j.inffus.2024.102748>
- Liu Y, Deng Q, Zeng Z et al (2025b) A hybrid optimization framework for age of information minimization in uav-assisted mcs. *IEEE Trans Serv Comput* 18(2):527–542. <https://doi.org/10.1109/TSC.2025.3528339>
- Ma W, Wang D, Song Y et al (2025) Trapnet: model inversion defense via trapdoor. *IEEE Trans Inf Forensics Secur*. <https://doi.org/10.1109/TIFS.2025.3560557>

- Misra V, Reddy JSK, Chimalakonda S (2020) Is there a correlation between code comments and issues? an exploratory study. In: Proceedings of the 35th Annual ACM symposium on applied computing, Brno, Czech Republic, SAC '20, pp 110–117
- Neamtii I, Foster JS, Hicks M (2005) Understanding source code evolution using abstract syntax tree matching. In: Proceedings of the 2005 international workshop on Mining software repositories. St. Louis, Missouri, MSR '05, pp 1–5
- Nguyen V, Le T, De Vel O et al (2021) Information-theoretic source code vulnerability highlighting. In: 2021 International Joint Conference on Neural Networks (IJCNN). IEEE, pp 1–8
- Ni C, Guo X, Zhu Y et al (2023a) Function-level vulnerability detection through fusing multi-modal knowledge. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, Echternach, Luxembourg, ASE '23, pp 1911–1918
- Ni C, Yin X, Yang K et al (2023b) Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. San Francisco, CA, USA, ESEC/FSE 2023, pp 1611–1622
- Peng T, You W, Guan K et al (2024) Privacy-preserving multiobjective task assignment scheme with differential obfuscation in mobile crowdsensing. *J Netw Comput Appl* 224:103836. <https://doi.org/10.1016/j.jnca.2024.103836>
- Pornprasit C, Tantithamthavorn CK (2022) Deeplinedp: towards a deep learning approach for line-level defect prediction. *IEEE Trans Software Eng* 49(1):84–9. <https://doi.org/10.1109/TSE.2022.3144348>
- Prystawski B, Li MY, Goodman ND (2023) Why think step by step? reasoning emerges from the locality of experience. In: Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23
- Qiu F, Liu Z, Hu X et al (2024) Vulnerability detection via multiple-graph-based code representation. *IEEE Trans Soft Eng*. <https://doi.org/10.1109/TSE.2024.3427815>
- Raffel C, Shazeer N, Roberts A et al (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res* 21(140):1–67
- Roziere B, Gehring J, Gloeckle F et al (2023) Code llama: Open foundation models for code. [arXiv:2308.12950](https://arxiv.org/abs/2308.12950)
- Russell R, Kim L, Hamilton L et al (2018) Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, pp 757–762
- Soydaner D (2022) Attention mechanism in neural networks: where it comes and where it goes. *Neural Comput Appl* 34(16):13371–13385. <https://doi.org/10.1007/s00521-022-07366-3>
- Steenhoek B, Rahman MM, Jiles R et al (2023) An empirical study of deep learning models for vulnerability detection. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, Melbourne, Victoria, Australia, ICSE '23, pp 2237–2248
- Svyatkovskiy A, Zhao Y, Fu S et al (2019) Pythia: Ai-assisted code completion system. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining. Anchorage, AK, USA, KDD '19, pp 2727–2735
- Tian Z, Tian B, Lv J et al (2024) Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding. *Expert Syst Appl* 238:12186. <https://doi.org/10.1016/j.eswa.2023.121865>
- Tipirneni S, Zhu M, Reddy CK (2024) Structocoder: structure-aware transformer for code generation. *ACM Trans Knowl Discov Data* 18(3):1–2. <https://doi.org/10.1145/3636430>
- Uddin MP, Xiang Y, Hasan M et al (2025) A systematic literature review of robust federated learning: issues, solutions, and future research directions. *ACM Comput Surv* 57(10):1–6. <https://doi.org/10.1145/3727643>
- Wan Y, Qu Y, Ni W et al (2024) Data and model poisoning backdoor attacks on wireless federated learning, and the defense mechanisms: a comprehensive survey. *IEEE Commun Surv Tutor* 26(3):1861–1897. <https://doi.org/10.1109/COMST.2024.3361451>
- Wang H, Ye G, Tang Z et al (2020) Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans Inf Forensics Secur* 16:1943–1958. <https://doi.org/10.1109/TIFS.2020.3044773>
- Wang Y, Li H (2021) Code completion by modeling flattened abstract syntax trees as graphs. In: Proceedings of the AAAI conference on artificial intelligence, pp 14015–14023
- Wang Y, Wang W, Joty S et al (2021) Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 conference on empirical methods in natural language processing, pp 8696–8708
- Wei J, Tay Y, Bommasani R et al (2022) Emergent abilities of large language models. [arXiv:2206.07682](https://arxiv.org/abs/2206.07682)
- Wen XC, Chen Y, Gao C et al (2023) Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, Melbourne, Victoria, Australia, pp 2275–2286

- Wen XC, Gao C, Gao S et al (2024a) Scale: constructing structured natural language comment trees for software vulnerability detection. In: Proceedings of the 33rd ACM SIGSOFT International symposium on software testing and analysis, Vienna, Austria, pp 235–247
- Wen XC, Gao C, Ye J et al (2024b) Meta-path based attentional graph learning model for vulnerability detection. *IEEE Trans Software Eng* 50(3):360–37. <https://doi.org/10.1109/TSE.2023.3340267>
- Wu Y, Zou D, Dou S et al (2022) Vulcnn: An image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, Pennsylvania. ICSE '22, pp 2365–2376
- Xiao P, Xiao Q, Zhang X et al (2024) Vulnerability detection based on enhanced graph representation learning. *IEEE Trans Inf Forensics Secur*. <https://doi.org/10.1109/TIFS.2024.3392536>
- Yamaguchi F, Golde N, Arp D et al (2014) Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE symposium on security and privacy. IEEE, pp 590–604. <https://doi.org/10.1109/SP.2014.44>
- Zhang J, Wang X, Zhang H et al (2019) A novel neural source code representation based on abstract syntax tree. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, Montreal, Quebec, Canada, pp 783–794
- Zhang J, Wang X, Zhang H et al (2020) Retrieval-based neural source code summarization. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, Seoul, South Korea, pp 1385–1397
- Zhang J, Pan L, Han QL et al (2021) Deep learning based attack detection for cyber-physical system cyber-security: a survey. *IEEE/CAA J Autom Sin* 9(3):377–39. <https://doi.org/10.1109/JAS.2021.1004261>
- Zhang J, Liu Z, Hu X et al (2023a) Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Trans Software Eng* 49(8):4196–421. <https://doi.org/10.1109/TSE.2023.3286586>
- Zhang S, Wang Y, Luo E et al (2023b) A traceable and revocable decentralized multi-authority privacy protection scheme for social metaverse. *J Syst Architect* 140:102899. <https://doi.org/10.1016/j.sysarc.2023.102899>
- Zhang S, Liu Q, Wang T et al (2024a) Fsair: fine-grained secure approximate image retrieval for mobile cloud computing. *IEEE Internet Things J* 11(13):23297–2330. <https://doi.org/10.1109/JIOT.2024.3384458>
- Zhang S, Pan Y, Liu Q et al (2024b) Backdoor attacks and defenses targeting multi-domain ai models: a comprehensive review. *ACM Comput Surv* 57(4):1–3. <https://doi.org/10.1145/3704725>
- Zhang S, Chen W, Li X et al (2025a) Apbam: adversarial perturbation-driven backdoor attack in multimodal learning. *Inf Sci* 700:12184. <https://doi.org/10.1016/j.ins.2024.121847>
- Zhang S, Zhang L, Peng T et al (2025b) Vadp: Visitor-attribute-based adaptive differential privacy for iomt data sharing. *Computers & Security* p 104513. <https://doi.org/10.1016/j.cose.2025.104513>
- Zhao Y, Qu Y, Xiang Y et al (2024) A comprehensive survey on edge data integrity verification: fundamentals and future trends. *ACM Comput Surv* 57(1):1–3. <https://doi.org/10.1145/3680277>
- Zhou W, Zhu X, Han QL et al (2025) The security of using large language models - a survey with emphasis on chatgpt. *IEEE/CAA J Autom Sin* 12(1):1–2. <https://doi.org/10.1109/JAS.2024.124983>
- Zhou Y, Liu S, Siow J et al (2019) Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Proceedings of the 33rd International conference on neural information processing systems. Red Hook, NY, USA, pp 10197–10207
- Zhu X, Wen S, Camtepe S et al (2022) Fuzzing: a survey for roadmap. *ACM Comput Surv* 54(11s). <https://doi.org/10.1145/3512345>
- Zhu X, Zhou W, Han QL et al (2025) When software security meets large language models: a survey. *IEEE/CAA J Autom Sin* 12(2):317–334. <https://doi.org/10.1109/JAS.2024.124971>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Shaobo Zhang^{1,2}  · Qianzhi Wang^{1,2} · Qin Liu³ · Entao Luo⁴ · Tao Peng⁵

✉ Shaobo Zhang
shaobozhang@hnust.edu.cn

Qianzhi Wang
wqz@mail.hnust.edu.cn

Qin Liu
gracelq628@hnu.edu.cn

Entao Luo
luoentaohuse@163.com

Tao Peng
pengtao@gzhu.edu.cn

¹ Sanya Institute of Hunan University of Science and Technology, Sanya 572024, China

² School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China

³ College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China

⁴ School of Electronics and Information Engineering, Hunan University of Science and Engineering, Yongzhou 425199, China

⁵ School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou 510006, China